
Replacing the Android Phone's Camera With a Virtual Camera

Snapchat Masking & Hacking

Zoe J.D.M. Spyrou

Student Number: 452280

Fontys HBO-ICT

Semester 4 OL

Table of Contents

- [Snapchat Masking & Hacking](#)
- [Context](#)
- [Objective](#)
- [Scope](#)
- [Research Methodology](#)
- [Vocabulary](#)
- [Process](#)
 - [Initial Attempts](#)
 - [USB Camera Approach](#)
 - [Virtual Camera](#)
 - [Debugging the VCam module](#)
 - [The Solution: MediaPlayer to VideoToFrames](#)
 - [Installing the module on the device](#)
 - [Implementing GitHub workflow](#)
 - [System Context Diagram](#)
 - [Container Diagram](#)
 - [Component Diagram](#)
 - [CI Workflows Diagram](#)
 - [Next steps](#)
 - [Risks](#)
 - [Android OS Update Risks](#)
 - [Snapchat Application Update Risks](#)
 - [Module Maintenance Risks](#)
 - [Operational Risks](#)
 - [Sources](#)

Date	Task
27/11/2025	Uploaded 1st version to Canvas + LSPosed Module Code
08/12/2025	Referenced pipeline + added images of module + explained how to install module
11/12/2025	Diagrams + risks
16/12/2025	Uploaded 2nd version to Canvas
19/12/2025	Added CI Workflows diagram

Introduction

The police uses Snapchat to talk to suspected individuals, through fake/controlled accounts portraying to be underage children, using AI generated images. The goal of our project is to find a way to mask the activity of the police and extract the IP address of the suspects, so that they can then use it to find out the location of the suspect.

Masking is defined as:

- Make it appear as if messages sent on Snapchat are being sent on a mobile device, not a browser, as Snapchat notifies users from where messages are being sent through
- Images that are being sent need to appear as images that were taken through Snapchat, and not as images directly uploaded through a camera roll, as those appear different in Snapchat
- Making an account look relatively active (snap score not 0, aka not 0 messages sent in total, as it appears that the account has not been used yet)
- Not notifying the suspect of screenshots taken of the conversation, needed for collection of evidence

All research and testing is done on accounts that are either owned by us (the team that is working on the project) or the stakeholders (police Rotterdam, Fontys coaches). No non-consenting parties are used in testing/experimenting. These efforts are not to be used in any scope outside the project, as that goes against the policy of Snapchat. All evidence found is to be kept between the Stakeholders and us.

Context

In this document I demonstrate how to use the Android's built-in camera to bypass Snapchat's "media upload" tag, which shows up when sending images that are stored on the device's camera roll, as opposed to being taken in the app itself.

Moreover, methods are developed to make this a user-friendly solution for the police.

Objective

Send images that appear "just-taken" in Snapchat. Make it a streamlined process for the police, with a simple to navigate front-end.

Scope

LSPosed module and user-friendly front-end.

Research Methodology

Based on the DOT-Framework, the following methodologies have been utilized:

Method	What/How/Why
Available Product Analysis	Researched available solutions/modules.
Multi-Criteria Decision Making	Same idea as available product analysis, leaning more on the comparison between the tools researched.
Literature Study	Reviewed a lot of online documentation and read about ways to use tools included in my testing, as well as feats of other people who tried to do something similar, including Snapchat and other social media applications.
Expert Interview	Discussed how effective this solution is with expert Tom Meulenstein and discussed about ways to manage the project for the future.
Stakeholder Analysis	Communicated current progress to the Stakeholder Ronald Beljaars, to ensure the project is on the right track.

Vocabulary

HAL

"A *hardware abstraction layer (HAL)* is type of abstraction layer with a standard interface for hardware vendors to implement. A HAL allows hardware vendors to implement lower-level, device-specific features without affecting or modifying code in higher-level layers." (*Hardware Abstraction Layer (HAL) Overview*, n.d.)

In short, it is the native vendor implementation that talks to the physical camera sensor.

Camera2

"Camera2 is the low-level Android camera package that replaces the deprecated Camera class. Camera2 provides in-depth controls for complex use cases, but requires you to manage device-specific configurations. For most developers, we recommend the CameraX Jetpack library." (*Camera2 Overview*, n.d.)

In short, it is a public framework API that applications use to talk to the Camera HAL.

CameraX

"CameraX is a Jetpack library, built to help make camera app development easier. For new apps, we recommend starting with CameraX. It provides a consistent, easy-to-use API that works across the vast majority of Android devices." (*CameraX Overview*, n.d.)

In short, it is a Jetpack library that wraps Camera2 to make common camera tasks much simpler.

Surface

"*Handle onto a raw buffer that is being managed by the screen compositor. A Surface is generally created by or from a consumer of image buffers (such as a SurfaceTexture, MediaRecorder, or Allocation), and is handed to some kind of producer (such as OpenGL, MediaPlayer, or CameraDevice) to draw into.*" (*Surface Overview*, n.d.)

In short, it is a an API that handles the way raw image data is delivered from the camera to various apps. This can happen in several methods, either with continuous video stream (MediaPlayer), graphics rendering (OpenGL), etc.

Process

Initial Attempts

Prior to working towards a solution, some research had been done on previous attempts from other users. The latest working solutions included the use of "OVF Editor" or specific filters on Snapchat, such as "Camera Roll". These solutions worked last around 2020, with no recent working solutions found easily online, that were ready-to-use, plug and play. Snapchat attaches the "media upload" tag to discourage users from passing older or edited images as new, to maintain the integrity and authenticity of the app. I had attempted to recreate the above mentioned, with no success. "OVF" is just an image editing application and the "Camera Roll Filter" is as it implies, is just a filter, that still attaches the upload tag to the image.

USB Camera Approach

After those attempts, I wanted to see how the phone would behave to an external camera. I used a standard USB webcam and connected it to the Android device with a USB-A to USB-C hub, also referred to as an On-The-Go adapter (OTG adapter). To "read out" the camera stream however, another camera app was needed, such as "USB Camera" from the Play Store. The idea behind this was the following: Connect an external camera to the Android device, use that camera instead of the built-in camera, and eventually replace the footage of that camera with pre-loaded images from a USB device. I quickly came to the realization though that this was not feasible, for 3 reasons:

1. Other camera apps are just a "viewer" for the built-in camera the phone has.
2. Changing the default camera application was not possible for Android 13 (The Android version running on the testing device).
3. Changing the default app would have no influence whatsoever on what camera Snapchat uses, as Snapchat communicates directly with the built-in camera hardware of the phone, and not with whatever default app is set on the device.

The solution would have to work with the actual hardware that was on the device or on the camera's HAL if it was going to work.

Virtual Camera

An LSPOSED module was tried, named VCam (虚拟摄像头 *Virtual Camera*, n.d.). This module was found through the Xposed Module Repository, (*Browse Modules*, n.d.). As opposed to other modules that did not mention anything about hardware or built-in phone directories, this one mentioned `Camera1` , making it seem promising.

Unfortunately, this module did not exactly work, though it remained promising, as Snapchat did notice it, in the sense that I got an unexpected error notice and the camera stream was blacked out, meaning that something indeed was going on with the camera, just not exactly what was needed yet.

The toast messages that popped up on the bottom of the screen, as well as when the phone was connected through `adb` on the computer, were all in Chinese.

Debugging the VCam module

To start the debugging process, I cloned the repository and translated all toast messages to English, with the help of AI. Then, I tried to recompile the module, and faced a lot of Java dependency issues in Android Studio. I updated the Java version in the module and matched it to the Java version on my system, and managed to compile it. From there, I had pushed the module back to the rooted phone, so I could retrace my steps, this time, at least being faced with English errors.

From those errors, I came to the conclusion that the encoding the module used for the images, was incorrect. While the encoding was not an issue for Instagram, Snapchat was not accepting the stream as a correct camera input.

Explaining this further: the Surface API delivers raw image data in different formats depending on what the app expects. In the initial module, the camera stream was delivered using `MediaPlayer` , sending a continuous stream of data to the virtual camera. However, Snapchat expects individual JPEG photo frames, not a video stream. The solution required using a custom decoder called `VideoToFrames` instead, which processes video into individual frames that Snapchat can recognize.

The Solution: MediaPlayer to VideoToFrames

The root cause was identified: Snapchat expects individual JPEG photo frames, not continuous video streaming. The fix involved modifying the VCam module to:

- Detect when an app uses JPEG format (code 256)
- Switch from MediaPlayer (continuous streaming) to VideoToFrames (frame-by-frame delivery)
- Apply proper surface classification for JPEG ImageReaders

This check allows the module to deliver video the way each app expects it. While the module is now fixed for Snapchat, it remains compatible with apps like Instagram that use continuous video streaming.

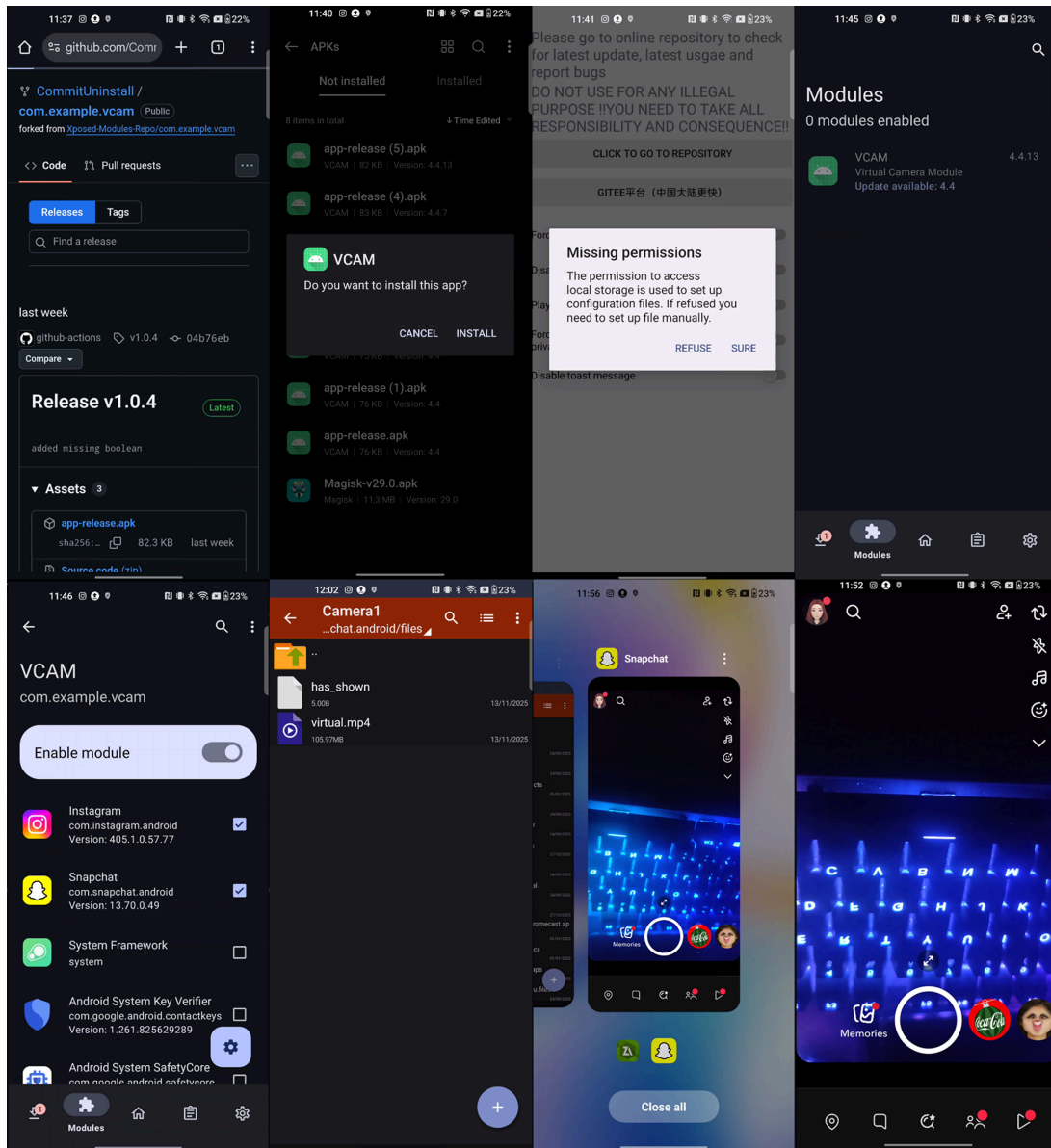
Installing the module on the device

Assuming the phone is configured as it has been in previous documentation (rooted with Magisk, LSPosed module installed through Magisk, device integrity checked), titled "[The journey of a Snapchat message - Traffic Capture](#)", which will be noted as well in the final technical guide, the installation is as follows:

1. Download the latest release of the package (app-release.apk file) through GitHub (CommitUninstall, n.d.)
2. Within the phone's file manager tap on the downloaded file to install it
3. Open the App and accept any permissions
4. LSPosed will show a pop up notification, saying that it detected a new module that has not been installed yet. Tap on the popup. If the popup is missed, scroll down your notification manager, find the LSPosed notification, and enter the menu. LSPosed can only be accessed through the notification center, not through an application directly on the phone.
5. Enable the VCAM module and by clicking on it, and clicking once again on "Enable Module". In that window, scroll down and activate Snapchat. This tells the module that the target is Snapchat
6. For this step, a file manager with root access is needed. For this setup, I am using ZArchiver. Open Snapchat once, for the module to read the app and gather all the information it needs. Snapchat will show an error, stating that there is something wrong with the camera.
7. In the following directory, place the virtual stream (the .mp4 file that will serve as the footage to override the camera):
"/storage/emulated/0/Android/data/com.snapchat.android/files/Camera1/ ". The video should have this exact naming: " virtual.mp4 ".
8. Fully close Snapchat if it still open, and reopen it.

9. At this point, you might still receive an error regarding the camera. The fix is to just expose all open apps, and then re-enter Snapchat without closing it. Now both the front and back facing camera are using the virtual stream, and videos as well as pictures can be taken as one would normally use Snapchat.

Here are the steps visualized, with screenshots of the installation progress.



guide.png

Implementing GitHub workflow

The module has been forked from the main repository, (虚拟摄像头 *Virtual Camera*, n.d.). After all changes had been done, and the app was manually imported to the phone, using Android Studio, with success, it was time to streamline the workflow, ergo GitHub workflows.

The purpose of that, was to streamline the installation process, as not everyone wants to build the project from scratch, as Android Studio and Java are known to be finicky with dependencies. Moreover, it is much easier to just download a version that is pre-built and quickly install it on your phone, like you would with any other APK.

I had decided on two Workflows, one for testing if the package could be built without errors, and one for stable releases with proper versioning, so that I could actually feature the module on the home page of the repository.

This is where I learned that feature releases had to be signed, as I could not install the application on my phone. I kept getting errors that the app I was trying to install was not signed. The process of signing the image was easy. I generated a keystore, using the following command: `keytool -genkey -v -keystore vcam-release-key.keystore -alias vcam -keyalg RSA -keysize 2048 -validity 10000`. After this, I had to convert the binary file generated to a base64 file, as GitHub secrets only store text, `base64 -w 0 vcam-release-key.keystore > keystore.txt`.

From there on, I added the required data to my GitHub Actions secrets for the specific repository:

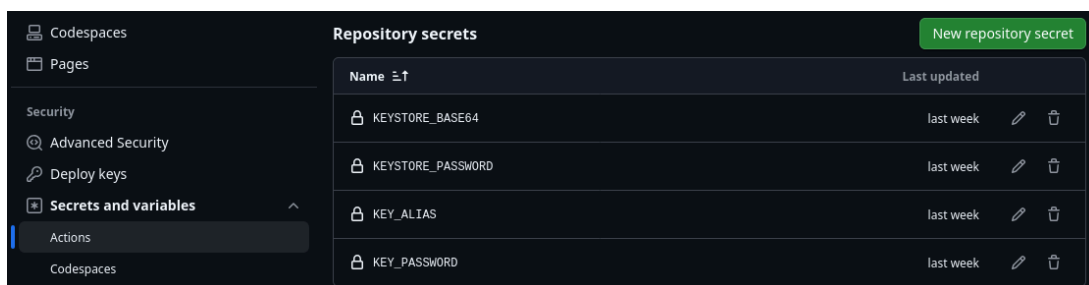


image-88.png

In my release Workflow, I had to point to the GitHub secrets, as the signing process would happen in the Workflow, after Gradle assembles the release.

The signing process serves several critical security purposes: it cryptographically proves the app's origin and authenticity, prevents malicious actors from distributing modified versions of the app, and ensures that only updates signed with the same key can replace the installed version. If the signing key is lost, it becomes impossible to update the app, even for the original developer, as Android will reject any updates signed with a different key.

I chose to only sign the release version, as the testing workflow produces debug APKs that are automatically signed with a temporary debug certificate by Android. Debug APKs are intended for developers who will test and modify the app, so the added security of release signing is unnecessary. This

separation also keeps the signing credentials out of the continuous integration pipeline, reducing the risk of accidental exposure.

System Context Diagram

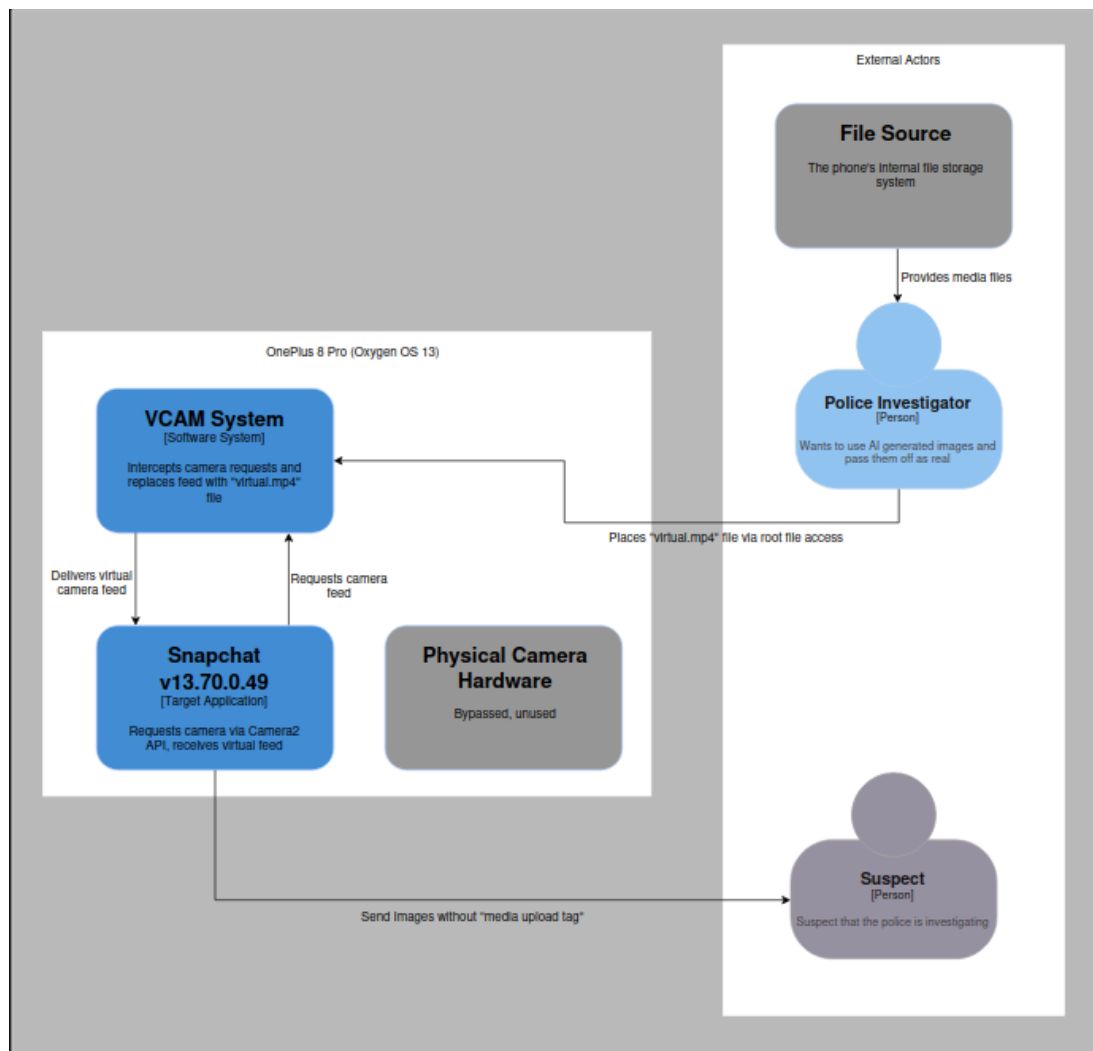


image-90.png

Container Diagram

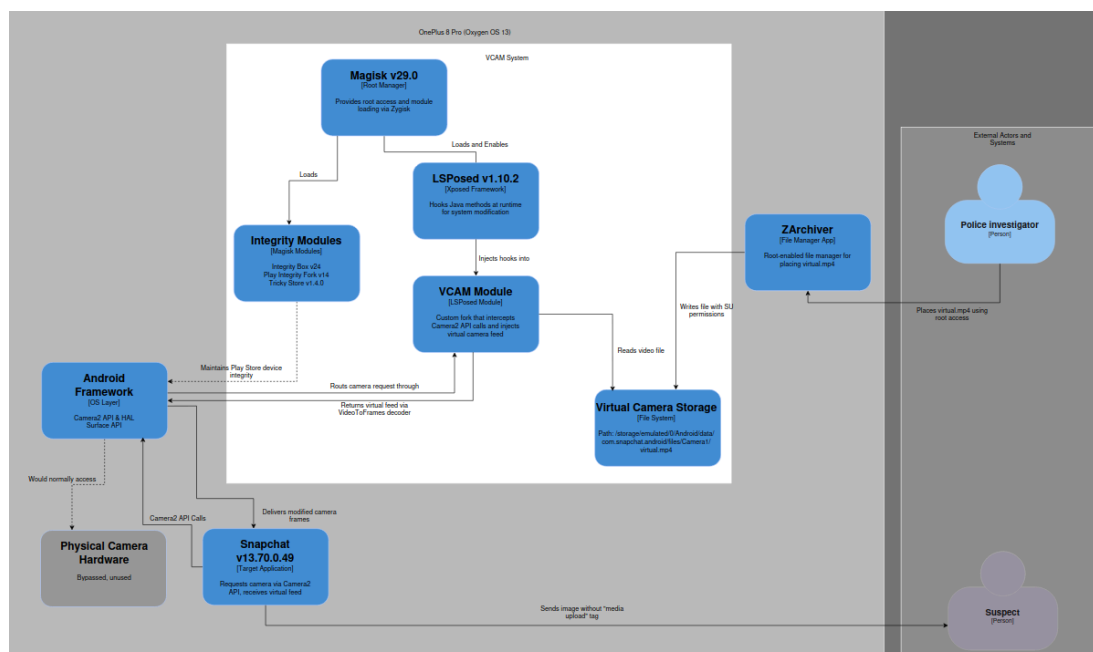


image-92.png

Component Diagram

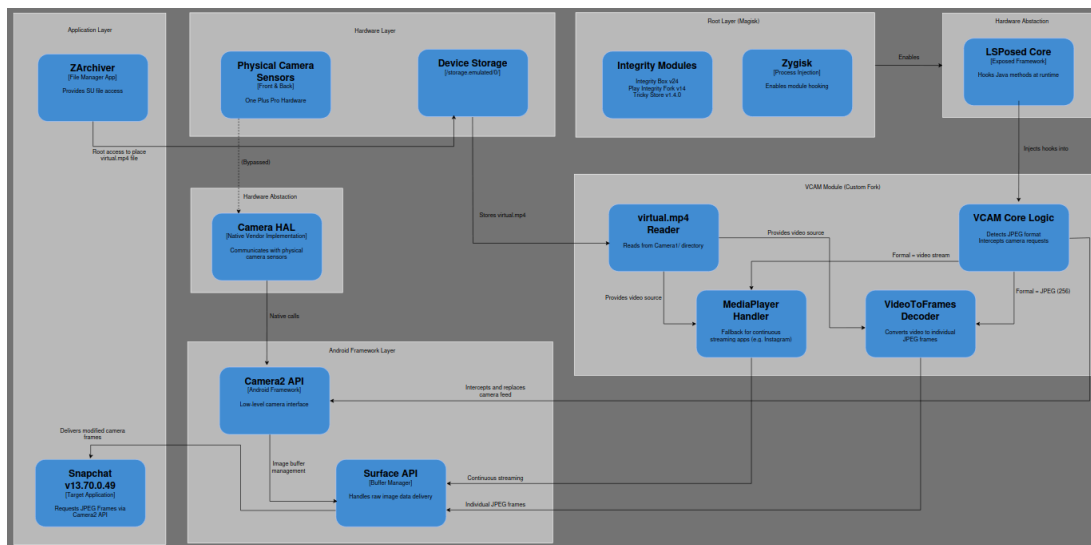


image-93.png

CI Workflows Diagram



image-105.png

Next steps

While this solution works in a slow paced environment, for a person that possesses some technical knowledge, this solution is not fit for the police. The police agents working with cases that require fake imagery to be sent, work in a fast paced setting that requires them responding instantly at times, or the potential suspect will cut contact if they suspect something weird is going on each time an image is being sent. Moreover, it cannot be expected that all agents pose the skills to modify root directories of an Android phone.

That is why I want to now create an interface that allows the uploading of images directly from the computer they work on, to the phone that is being used during the investigation. This way, they will interact with a user friendly front end that does not require technical knowledge, while still being able to use this module. This will be validated by the Stakeholder Ronald, as he is our primary stakeholder in this project. Based on his opinion on a low to medium fidelity design, a targeted UX will be designed.

Based on a talk with infrastructure expert Tom Meulenstein, the following steps will also be taken, to ensure the product stays up-to date and future relevant:

- A pipeline will be set up for the LSPosed Module, so that changes can be easily rolled back in case something goes wrong. Testing can be included, to ensure the product meets certain criteria. (Update 08/12/2025: The pipeline has been built and the APK can be downloaded (CommitUninstall, n.d.))
- The front-end mentioned in the next steps, should be containerized, in for example, Docker. This will ensure that the solution works seamlessly without dependency issues for the Stakeholder.

The stakeholder has moreover been asked if the modified version of the module can be uploaded publicly back on Github. My reasoning behind this was:

- The module was found on Github. I had modified it to behave in certain different ways, and fixed some dependency issues.
- If the module's fork is to be made public, future developers can work on this fork and thus, ensure it stays up to date, with for example later Android versions, which could help the police in return.

Any publications made on Github will not include anything that traces back to the police, to ensure the police's activities stay secretive.

Risks

As of yet, I was not able to test what happens when either the phone needs to update. Moreover, I have not updated Snapchat yet to verify continued compatibility.

Android OS Update Risks

The OnePlus 8 Pro reached its end of life in April 2024 (*OnePlus 8 Pro Latest Android Version Info*, n.d.). The device currently runs Android 13, which was its final major update released in November 2022.

Key concerns:

- Future Android versions may require updated LSPosed/Magisk versions with different behaviors
- Major Android updates could break module compatibility entirely

Mitigation:

Maintain current Android 13 configuration, test module on newer devices before deployment, document known-working version combinations.

Snapchat Application Update Risks

Snapchat receives frequent updates that could detect or break the VCAM module's functionality through enhanced security checks, API changes, or improved validation of camera inputs. Detection could result in account bans, compromising investigations.

Mitigation:

Disable automatic updates for Snapchat v13.70.0.49, maintain separate testing device for update evaluation, establish rollback procedures.

Module Maintenance Risks

The original VCam module is no longer actively maintained, and as a fork, all bug fixes and compatibility updates depend on continued internal maintenance by the police or repository contributors. Module crashes can occur when not properly optimized for specific devices or Android versions.

Mitigation:

Maintain comprehensive documentation of modifications, establish bug

reporting protocol with stakeholders.

Operational Risks

The installation process requires technical expertise (root access, LSposed configuration, specific directory structures), creating risk of user error during critical investigations. Additionally, integrity modules may fail with future Google Play Services updates.

Mitigation:

Develop user-friendly front-end interface as priority, create automated testing scripts, maintain backup device with known-working configuration.

Sources

Camera2 overview. (n.d.). Android Developers.

<https://developer.android.com/media/camera/camera2>

CameraX overview. (n.d.). Android Developers.

<https://developer.android.com/media/camera/camerax>

Hardware abstraction layer (HAL) overview. (n.d.). Android Open Source Project. <https://source.android.com/docs/core/architecture/hal>

虚拟摄像头 virtual camera. (n.d.). Xposed Module Repository.

<https://modules.lsposed.org/module/com.example.vcam/>

Browse modules. (n.d.). Xposed Module Repository.

<https://modules.lsposed.org/>

Surface overview. (n.d.). Android Developers.

<https://developer.android.com/reference/android/view/Surface>

CommitUninstall. (n.d.). Releases · CommitUninstall/com.example.vcam.

GitHub. <https://github.com/CommitUninstall/com.example.vcam/releases/>

Perry, C. (2017, June 4). *What is Android keystore file, and what is it used for?* Stack Overflow. <https://stackoverflow.com/questions/6849558/what-is-android-keystore-file-and-what-is-it-used-for>

OnePlus 8 Pro latest Android version info. (n.d.). Android Update Tracker.

<https://www.androidupdatetracker.com/p/oneplus-8-pro>